

KOMODOR

Rules of Engagement

SE Operating Manual for Komodor POCs

1. About this manual

This is your operational manual for every Komodor POC. It defines what you, the SE, are responsible for at each stage and gives you the checklists to execute against. Work through each section in order.

How this manual is organized:

- **Section 1** — Overview of responsibilities
- **Section 2** — Preflight checklist: what to complete before the POC starts.
- **Section 3** — During the POC: kickoff training and the steady-state cadence to run while the POC is live.
- **Section 4** — Klaudia: how it must be used in the process, and why.
- **Section 5** — Securing the technical win: how to close the technical evaluation.
- **Section 6** — When something breaks: how to respond, isolate, and escalate.
- **Section 7** — Business cost: what skipping these steps costs the business and your OTE.
- **Section 8** — Golden state: the reference configuration to validate against.
- **Sections 9–10** — Known tooling constraints and open items to be completed.

1.1 What you are responsible for

You are the single accountable owner for the POC and the overall technical engagement, end to end. Ownership covers everything technical the customer experiences — not only the Komodor agent — and also the analysis, review, summary, and storytelling that come out of it. Nothing is handed off. Agent install and validation is a critical part of this responsibility and is highlighted in Sections 2 and 3.2. In full, you are responsible for:

1. **Creating success criteria that are differentiated, negotiated and agreed-to** — It is critical for a Komodor SE to have an opinion on what the prospect should evaluate and how it solves their current challenges and/or helps them accelerate their company goals (Revenue, developer productivity, etc). SE is also responsible for ensuring we and the prospect have a clear understanding of what persons or groups are responsible for validating each individual success criteria.
2. **Agent install & validation (critical)** — providing detailed installation and deployment guidance per the customer's preferred method (Helm or Argo CD/Flux etc.), confirming the agent was installed and deployed correctly against the values you specified, and validating that the Komodor platform is fully operational and behaving as intended. Detailed in Section 2 (preflight) and Section 3.2 (steady state).
3. **Training and Enabling the prospect/customer-** SE is responsible for training and enabling the prospects such that the prospect can effectively use the product.
4. **Ongoing technical health & core functionality** — keeping the platform correct and operational throughout the POC, and reviewing that all core capabilities are working and visible (workload/infra health, cascading failures, cluster/service counts, pod logs and exec). Detailed in Section 3.
5. **Demonstrated value — regularly running RUM sessions and AI-investigator to find interesting examples and areas of frustration that you can address.** Coming prepared to every POC sync call and actively presenting value and impact, with worked examples from the platform (the Rigging report), quantified before/after and proactively addressing any product/UX friction. **Detailed in Section 3.6.**
6. **Communicating Value** — consistently communicating updates on progress, key features and capabilities, what's impactful, what the prospect finds compelling, product gaps and feature requests.
7. **Securing the technical win** — closing the technical evaluation by confirming the success criteria were validated and by whom, capturing sentiment and product feedback, and explicitly asking for a commercial

recommendation. Anything other than an unconditional yes triggers in-the-moment discovery. Detailed in Section 5.

8. **Issue escalation to Product / R&D** — owning the identification, the escalation, and driving any genuine agent or product issue/feedback through to resolution. The escalation is not a handoff; you stay on it until the issue is addressed in totality. Detailed in Section 6.
9. **Risk visibility** — proactively calling out and clearly defining risks as early as possible, rather than waiting for them to surface.
10. **Deal ownership and communication**

One-line standard

Once an opportunity is declared qualified by the sales team and there is a high propensity for a Komodor POC to happen, you own everything technical about it — the technical validation criteria, the tech stack, verifying an operational agent, the analysis, the customer-facing story, and the close — end to end, every time, no exceptions. While product, RND, support, and others may be responsible for individual tasks, the SE is the quarterback and is responsible for ensuring everything is complete, checked off and followed through upon.

2. Preflight Checklist

Preflight is centered on agent install, validation, and connectivity — the critical foundation for the rest of the engagement. Without a stable foundation, POC success is significantly less likely. Complete every item below before declaring the agent ready for the POC. Each item is your responsibility to perform and to verify.

Prerequisite — complete and validate before any other step

Before performing any step in this checklist, confirm that all steps in the [Komodor Evaluation Pre-Flight Check](#) are completed and validated fully.

2.1 Agent staging to the customer's internal registry

While we prefer to use public, official Komodor registries, many enterprise customers will not pull images from public registries. In this case, the customer will need to stage the Komodor agent images into their internal registry ahead of deployment. Never assume your counterpart on the prospect side knows what they are doing.

- ☐ Confirm the target registry is Docker Registry HTTP API V2 compliant (any V2-compliant registry is supported).
- ☐ Provide the full list of required Komodor agent images and tags (*see Open Items, 10*) and ensure the customer mirrors them into the customer registry.
- ☐ Have the customer verify image pull works from inside the target cluster using the customer's image pull secret / service account.
- ☐ Have the customer confirm image tags match the version you prescribed to install

2.2 Configuration validation — Helm and Argo CD

The agent must be validated against the customer's actual deployment method. The two supported paths are Helm chart and Argo CD (or any CD tool).

- ☐ For Helm deployments: review the rendered values against the published default Helm values and flag every override.
- ☐ For CD deployments: review the Application manifest / values overlay and flag every override versus default.
- ☐ Confirm no override changes a default value in a way that degrades or disables agent function (this is the exact failure this ROE exists to prevent).
- ☐ Record the final, as-deployed configuration so it can be compared to the golden state and re-verified later.

2.3 Default values must be communicated explicitly

A recurring source of failure is assuming the customer knows what the defaults are and why they matter. You must distinguish, in writing, between two categories:

Communicate, do not assume

- **Stated defaults** — values you have explicitly communicated to the customer as the recommended/default configuration, including the consequence of changing them.
- **Assumed knowledge** — anything you merely believe the customer “should know.” This category must be treated as a gap and converted into a stated default.

Rule: if a default matters to agent function, it must be a stated default. The customer must never be left to infer it.

- ☐ Provide the customer a written list of the default values that matter, with the impact of overriding each.
- ☐ Where the customer has overridden a default, confirm the change was intentional and verify it with the customer / end user before proceeding.

2.4 Deployment confirmation & operational validation

After install, you personally confirm the deployment and validate the platform — the agent being “installed” is not the same as the platform being “operational.”

- ☐ Confirm the agent installed and deployed correctly via the customer’s chosen method (Helm or CD).
- ☐ Validate that the Komodor platform is operational end to end: the agent is reporting, DaemonSet Pods are scheduled and running, data is flowing, and Klaudia and core workflows behave as intended for logs, metrics, resources, etc.
- ☐ Review the customer’s infrastructure, workloads, and services in Komodor, and spot-check agent capabilities against them to confirm the agent sees what it should.
- ☐ Run a Klaudia chat to validate the agent deployment — confirm Klaudia can reason over the live environment as expected (see also 4).
- ☐ Capture a baseline screenshot / export of the operational state to compare against later in the POC.

2.5 Regarding Cost Optimization

In addition to the above, the SE/SA is responsible for the correct and optimal configuration of the Komodor agent and its constituent components to ensure a successful cost optimization POC. The current cost optimization preflight checklist is located here: [Komodor Cost Optimization Preflight Checklist](#)

3. During the POC

Preflight gets the agent into a correct, operational state. Steady state keeps it there and broadens the focus to the full technical success of the engagement — core functionality, user activity, cost optimization, and demonstrated value. Maintaining a correct, operational agent remains a critical part of this stage (3.2), and you own it continuously — configuration drifts, customers change their environment, and an agent that was healthy on day one can quietly degrade. The duties below are a recurring cadence, not one-time tasks.

3.1 POC kickoff — training and enablement

Kickoff transfers the platform from “installed” to “being used.” A perfectly installed agent utilized by an untrained team will lose to a worse product that kicked off with great training and enablement. Run the training and enablement after the agent is installed but before you have allowed users to access the platform. The benefit is that the POC starts with participants having fresh and pointed knowledge on how to operate the Komodor platform instead of using the product how they see fit.

- ☐ Schedule and run the training and enablement with the customer’s evaluation team after the agent is installed but before you have allowed users to access the platform. The benefit is that the POC starts with participants having fresh knowledge on how to operate the Komodor platform.
- ☐ Walk through the platform’s core capabilities: workload/infra health, cascading failures, cluster and service views, pod logs and exec, and Klaudia (Section 4).

- ☐ Understand the prospect's day-to-day workflows and demonstrate how our product fits in.
- ☐ Confirm every member of the evaluation team has access and can log in. Encourage each to sign up themselves — this also triggers the signup alerts in 3.4.
- ☐ Confirm or formally agree the success criteria the team will evaluate against (these are what you will close back to in 5.1).
- ☐ Set the sync-call cadence (weekly is typical) and confirm attendees.
- ☐ Share self-service resources: documentation, key dashboards, and how to reach you with questions.
- ☐ Identify and capture the names of likely champions on the evaluation team — they are who you build relationships with throughout the POC.
- ☐ Introduce Komodor support and how to receive support when a question or issue arises.

Why kickoff training is non-negotiable

Komodor has no open-source, community, or freemium edition. You are the only path for the prospect's team to learn how to use the platform as it is intended. Without a kickoff that walks them through it, the evaluation team spends the POC fumbling instead of evaluating — and judges Komodor on whatever fragment of it they stumble into. Kickoff is also where the prospect first experiences the AI SRE on their running cloud native infrastructure (see 4).

3.2 Maintain a correct, operational agent

- ☐ Re-validate the agent against the golden state (8) on a set cadence — do not assume it remains correct. Use the Klaudia inventory / golden comparison (4) to do this efficiently.
- ☐ Re-confirm the platform is operational after any customer-initiated change to the cluster or environment (upgrades, policy changes, registry changes, scaling events).
- ☐ Keep the as-deployed configuration record current; update it whenever a value changes so the golden comparison stays meaningful.
- ☐ Watch for and triage anomalies early — a degraded agent caught on day two is recoverable; one caught at the readout is not.

Trigger: any environment change = re-validate

Whenever the customer changes their environment, you re-confirm agent configuration and platform health before treating any subsequent result as valid. An unverified change is the exact failure mode this ROE exists to prevent — it applies for the whole POC, not just at setup.

3.3 Review all core functionality

Actively review that the platform's core capabilities are working and visible — not just that the agent is connected. Cover, at minimum:

- ☐ Workload and infrastructure health.
- ☐ Cascading failures — confirm the platform surfaces them as expected.
- ☐ Cluster counts and service counts.
- ☐ Pod log fetch and pod exec.

Objection handler — pod logs & exec

Pod log fetch and pod exec are covered under the trial EULA, so no one should object to enabling them. Use this if a customer raises concerns.

3.4 Review user activity at least twice a week

Review how the prospect is actually using Komodor at least twice a week. The goal is to capture real user interactions and frustrations while there is still time to act on them. For every line item below, ask yourself “is this impactful to the prospect?”. If you cannot answer that question, find it out.

- ☐ Configure an alert in **#user-signups-paying** and **#self-serve-signups** for the prospect's email domain. Use each new sign-up as a trigger to run a RUM session and surface patterns of risk, opportunity, and friction.
- ☐ Datadog RUM — review before every sync call, and at least once a week regardless. Open Datadog RUM at app.datadoghq.com/rum/replay/search. Adjust the time-scope filter (top right) to your window (last 7d or 30d) and filter the query to the customer / prospect's account. Use the WHERE USERS DID filter to scope to specific user events. Focus on sessions where users open Klaudia, send a prompt, and act on the answer — those are the value moments. Pair each strong session with its Klaudia Dashboard entry for the under-the-hood view.
- ☐ Klaudia Dashboard — Komodor's internal tracker for every Klaudia session (prompts, tool calls, investigations, outcomes). Open at app.komodor.com/klaudia-dashboard/. (Access required — request it if you do not have it.) From the Session History tab, filter by account name to pull recent sessions, then filter the Flows field to RCA for root-cause sessions and ChatOnCluster / ChatOnResource / ChatOnSession for chats. For each session, note the actions Klaudia took, the evidence it pulled, and the outcome — and use this view to validate the RCA was deep and spot any issues in Klaudia's end-to-end flow. Pair with the RUM replay to see what the user saw alongside what Klaudia did.
- ☐ Heap — product analytics for Klaudia adoption: how many users, how often, which capabilities. Use it to talk about scale and stickiness, not individual sessions. Start at the Klaudia sessions dashboard (heapanalytics.com/app/env/3442179137/dashboard/Klaudia-361061) and browse the related dashboards — there is more than the example link. Segment to the customer / prospect and look for unique users, sessions per user, anomalies, WoW or MoM growth in active users, repeat usage by the same user, and which Klaudia capabilities they lean on most. Usage curves over time tell a stronger story than raw totals.
- ☐ Klaudia AI summary — use it to digest activity and surface what matters quickly.

3.5 Cost optimization simulation & reporting

Cost optimization is a core value driver and must be set up early — it depends on accumulated data, so it cannot be improvised on the day of a call.

- ☐ Be proactive in engaging the product team — do not start this on the day of your call.
- ☐ Allow enough data: 14 days is ideal. Product guidance is 7 days; add an extra week to absorb anomalous data.
- ☐ Run right-sizing and bin-packing simulations.
- ☐ Reporting: today this may require support; expect it to become self-service shortly.

3.6 Prove value during the POC

A POC is won by what gets actively presented to the customer, not by what the platform quietly finds. You come prepared to every sync call with worked examples and a quantified before/after — and walk the customer through the value and impact directly.

- ☐ Show up to every scheduled sync call prepared with worked examples from the platform (the Rigging report).
- ☐ Bring an informed opinion of how long each example would have taken the customer to do without Komodor.
- ☐ Validate that the Rigging report examples are genuinely impactful — capture the “juicy” ones (e.g. **cascading failures** and **best-practice violations**) as evidence for the readout.

Customer environment warning

When using Komodor or running Klaudia in the customer's live environment or impersonating a user, NEVER trigger 1-click remediation. These techniques are for demonstration and evidence capture only. Always exit impersonation immediately when done.

- ☐ Live demo on a real issue — open the customer's environment in Komodor and scan current health: failing services, recent events, active alerts. Either trigger an RCA on a real issue, or open Klaudia chat on a specific cluster / resource / session and ask the kind of question a user would ask. Pick an issue with

a clear story (recent deploy, config change, known incident) so the RCA reads as a strong narrative. Watch the flow end to end with the customer in the room — which evidence Klaudia pulls, how it reasons through the chain of events, and the final recommendation.

- ☐ Impersonate a user to capture customer-specific evidence — from Settings → Users, click Impersonate on a user from the account to browse Klaudia’s chat history exactly as they see it. Capture the full flow (prompt, investigation steps, final RCA) as evidence for sync calls and the technical-win readout. Cross-reference with the Klaudia Dashboard (3.4) for the under-the-hood view. Exit impersonation when done.

Why this matters

The usage signals from RUM and Heap, the Klaudia sessions in the dashboard, and the impactful examples in the Rigging report are the evidence that makes the business case at the readout. Arriving prepared, with quantified before/after and customer-specific screenshots, is part of owning the POC.

3.7 Own every mid-POC change

You own any change to the agent during the POC end to end — the same standard as install. Because the cluster is the customer’s, every change is communicated and confirmed, never silent (full mechanism in 4.3).

- ☐ Provide written instructions for any change you will make to, or request in, the customer environment.
- ☐ Obtain customer acknowledgement before the change is applied.
- ☐ Re-validate against golden state after the change and record the result.

3.8 Keep the customer informed

You own the customer-facing narrative throughout the POC, both for value created (rigging reports, before after scenarios, cost optimization potential) but especially when something is being investigated or escalated.

- ☐ Communicate agent / platform status to the customer at an agreed cadence.
- ☐ During any open escalation, keep the customer’s expectations current and aligned to the agreed response times (6.2).
- ☐ If an update is required, give one. Even if the update is “we’re still working on it.” Be transparent and genuine unless you have a valid reason.
- ☐ Surface configuration concerns proactively rather than waiting for the customer to notice a problem.

3.9 Weekly POC reporting to internal channels and Salesforce

At least once a week, you produce the analysis and summary of the POC — key findings, interesting events, and impactful sessions — and you make it visible to the team and to leadership. You own this analysis end to end; it is not raw material someone else packages. The output keeps the team aligned, feeds the deal record with intelligence for forecasting and escalation, and keeps the Rigging report current.

- ☐ Identify impactful sessions and events from the past week — Klaudia RCAs that moved the customer’s understanding, chat sessions that demonstrated value, moments that directly addressed a success criterion (5.1), and any events that surfaced risk or opportunity. Pull 1–3 strong examples.
- ☐ Capture your key findings — what you learned about the customer’s environment, what worked, what needs attention, and any risks or blockers.
- ☐ Post your weekly update to the per-account internal Slack channel (#internal-<customer>) — include the date, your key findings (2–4 bullets), strongest session links from the Klaudia Dashboard, and any follow-ups you are driving.
- ☐ Update the Salesforce POC record with the same findings and session links — this is your analysis on the deal record, so leadership, the AE, and forecasting see your work without chasing you for status.
- ☐ Feed the Rigging report — as impactful examples emerge from Klaudia sessions or customer discussions, capture them so the report stays current week to week and is ready for the readout (3.6, 5).

3.9.1 Reporting on key milestones in the POC

1. During key milestones, you are expected to provide an update, analysis and summary of the key milestone in the POC— success criteria alignment. training kickoff with number and types of users and impactful sessions — and you make it visible to the team and to leadership.
2. You own this analysis end to end; it is not raw material

4. Klaudia and Tooling best practices

Please follow [Klaudia SE enablement v2](#)

5. In-Product Self-Checks (Klaudia)

Beyond the manual preflight, you should use the product itself — specifically Klaudia — to report on agent versions and configuration wholesale across the POC environment. This turns validation from a one-time manual step into a repeatable, in-product check.

Why you must use Klaudia in the process

Komodor’s category claim is “AI SRE.” Klaudia is that AI SRE. Every time you use Klaudia in the POC workflow — to validate the deployment, inventory configs, summarize activity, or triage an anomaly — the customer sees that claim demonstrated live on their running cloud native infrastructure. They do not have to take our word for it; they watch it work. There is no stronger proof point we can offer in a POC.

Using Klaudia in your workflow also:

- Tests the surface the customer will actually use to judge the platform — not just the internal plumbing behind it.
- Scales SE validation across many clusters and agents (one prompt instead of an N-cluster manual audit).
- Compresses steady-state attention — RUM, Heap, and the live environment digested into a chat.
- Builds your fluency for customer demos: the prompts that work for you become the prompts that resonate with the customer.

If you do not use Klaudia, the customer never sees the AI SRE in action — and the entire category claim becomes a slide, not a demonstration.

4.1 Wholesale agent version & config reporting

The goal is a single, environment-wide view of every agent: which version is running where, and how each is configured.

- Report agent versions across the entire POC environment in one place, rather than cluster-by-cluster spot checks.
- Surface configuration per agent so overrides are visible without manually reading each Helm release or Argo Application.

4.2 Klaudia chat prompts to inventory & compare to golden state

Explore using Klaudia chat prompts to inventory the agent versions that are running and compare them against a golden, best-practices state.

- **Inventory prompt** — ask Klaudia to list all running agent versions and their key configuration values across the environment.
- **Comparison prompt** — ask Klaudia to compare the live inventory against the golden best-practices state and report drift / deviations.
- **Exception report** — have Klaudia highlight only the agents that deviate from golden, so you focus on what is wrong.

Note — feasibility to confirm

Using Klaudia chat prompts to drive inventory and golden-state comparison is proposed here as a capability to validate.

4.3 Driving change inside the POC environment

Where the inventory finds drift, you need a defined mechanism to drive the agent back to the correct state. Because the cluster is the customer's, this always means giving the prospect clear instructions for changes you will make to (or request in) their environment.

- ☐ Produce explicit, written change instructions describing exactly what will be changed in the customer environment and why.
- ☐ Obtain customer acknowledgement before applying any change to their environment.
- ☐ Re-run the Klaudia inventory / golden comparison after the change to confirm the agent now matches golden state.

Why you must ensure the Komodor Agent is correctly installed.

If you do not certify the agent installation, Klaudia may be working from an incomplete set of data, producing incomplete or lacking assessments. Accurate and complete data flowing from a customer's environment to Komodor is of utmost importance.

5. Securing the Technical Win

At the end of the POC, your job is to close the technical evaluation cleanly: confirm what was validated and by whom, capture sentiment and product feedback, and explicitly ask for a recommendation on a commercial relationship. Anything other than an unconditional yes is a discovery opportunity — work it in the moment, before the customer leaves the room.

5.1 Confirm the success criteria were validated

Success criteria should have been agreed with the customer up front. The closing conversation confirms they were met, and by whom.

- ☐ Walk the evaluation team through each agreed success criterion and confirm it was validated during the POC.
- ☐ Confirm by name who on the evaluation team validated each criterion. These names are your internal references and your champions going forward — capture them.

5.2 Listen for sentiment and product feedback

- ☐ Listen for any neutral sentiment and note it. Neutral is not positive — it usually signals an unspoken concern worth surfacing.
- ☐ If you get feedback on product gaps or enhancements: acknowledge and note for follow-up. Do not negotiate or commit in the room.

5.3 Ask for the commercial recommendation

Close with the direct question: **“Will you recommend a commercial relationship with Komodor?”**

If the answer is anything other than an unconditional yes — an outright “no,” or a “yes” with any conditions — do more discovery in the moment using the prompts in 4.4.

Standard

An unconditional yes is the only signal that the technical win is secured. Anything else is data, not a decision — surface it now, not after the call.

5.4 In-the-moment discovery by response

When the answer is not an unconditional yes, use the matching prompt below to keep the conversation going while the customer is still in the room. Capture the answer, the people named, and the conditions.

Response from the customer	What you ask next
"I'm not the person responsible for the decision."	"Who else needs to see that we have been able to prove the Komodor platform?" Capture the names and what each one needs to see.
Outright "no."	"What have you not seen out of this POC that gives you pause?"
Conditional yes — "we need budget approval."	"What's the budget approval process, and what does the approver need to see to approve it?"
Conditional yes — "yes if pricing works."	"What's the commercial structure or price point you'd need to see for this to work?"
Conditional yes — pending security / procurement / legal review.	"What's the procurement timeline, and is there anything in security or legal we should pre-empt now?"
Conditional yes — "I need to bring others along."	"Who else needs to be convinced, and what would convince them? Can we get them in a room together?"
Conditional yes — "not now / wrong timing."	"What needs to be true in your environment before this is the right time, and when would you want to start?"
Conditional yes — "yes if you can deliver feature X."	"Is X a deal-blocker, or a nice-to-have? If we had it today, would you sign? What's the next-best alternative until then?"
"We're going with [a competitor]."	"What gave them the edge? Where did Komodor not measure up in this POC?"
"We'll build this ourselves."	"What does the build look like in scope, headcount, and timeline — and what would need to be true for buying to make more sense than building?"
"ROI didn't land."	"What's the value threshold this would have needed to clear to justify the investment?"
"Champion has changed / no longer involved."	"Who is the right person to talk to now, and what context do they need to pick this up?"

5.5 What "complete" looks like

- ☐ Each success criterion has been explicitly validated, with named validators on the evaluation team.
- ☐ All neutral sentiment and product feedback is captured for follow-up.
- ☐ The commercial recommendation question was asked and answered.
- ☐ For anything other than an unconditional yes: in-the-moment discovery was completed, with named next people, next steps, and conditions captured.

6. When Something Breaks

Things break during POCs — that is the nature of the work. What separates a recoverable situation from a deal-killer is your response in the moment. This section first sets the response posture, then defines the operational steps: isolate the issue (6.1), and escalate through the right channel (6.2).

What is NOT acceptable when something breaks

When something goes wrong in a POC, the following responses are off the table:

1. Suffering in silence.
2. Lamenting that the software does not work.
3. Pointing to past experience that "should have" prevented this.

All three may be true. They are also completely useless to you doing your job. The job, when something breaks, is to isolate the issue and move it through the right path — not to commentate on it.

The productive response is: isolate (6.1), escalate by severity through the right channel (6.2), and keep the customer informed throughout (3.8).

6.1 SE-led isolation (before escalation)

You own first-line troubleshooting. The objective is to separate a customer-specific / environment issue from a genuine Komodor agent bug and attempt to reproduce the issue.

- ☐ Reproduce the issue on a non-prospect, not in POC cluster to determine whether it is customer-specific or product-wide.
- ☐ Confirm the configuration matches the golden state (8) so config is ruled out as the cause.
- ☐ Rule out environmental causes already validated in preflight (registry access, ports/egress, RBAC, proxies).
- ☐ Document reproduction steps, expected vs actual behaviour, agent version, and config diff before escalating.

Decision: config issue vs product bug

- **Resolves when config is corrected to golden** → configuration issue, owned and closed by you.
- **Persists with correct golden config, and reproduces on the internal test cluster** → genuine product/agent issue → escalate to R&D (6.2).

[Add specific symptoms / error patterns (incl. any Klaudia-surfaced signals) that should trigger escalation — to be filled in]

6.2 Getting support from Product / R&D

Once you have isolated a genuine product issue, follow these steps in order to escalate. Each severity has its own channel. The escalation is yours to drive end to end — opening the ticket and posting in the channel does not transfer ownership. You stay on it until Product / R&D resolve the issue and you have communicated the resolution back to the customer.

1. Troubleshoot on your own first when possible — isolate the issue per 6.1 before escalating.
2. Open a support ticket via Zendesk to track the issue. R&D will prioritize and triage the issue and also file a Clickup Bug as needed .
3. It's still your immediate responsibility to raise it in the channel that matches the severity (see table below).
4. Tag your immediate manager. For a critical POC, also tag **Ricardo Lupo**

Severity	Definition	Escalation path	Target response
P0	Something wrong that can impact all customers; a high-profile POC at risk (gauge by deal size, logo, and forecast); or a POC going sideways.	Raise in #p0-bugs ; open a support ticket; tag your immediate manager.	<u>ASAP</u>
P1	A platform degradation, or something wrong pervasively across the product.	Raise in #p1-bugs ; open a support ticket; tag your immediate manager.	<u>Next business day</u>

What you communicate to the customer

You own the customer-facing message during an escalation. Set expectations using the agreed target response times — never promise a turnaround the escalation path cannot support.

7. Business Cost — and What It Costs You

Skipping the steps in this manual is not a neutral shortcut. The cost compounds first in the business, and then lands directly in your OTE. POCs that stall or fail because of skipped steps do not close — and POCs that do not close do not pay commission.

Dimension	Cost to the business	What it costs you
POC timeline	Days burned re-installing, re-validating, and rebuilding trust inside a fixed evaluation window that cannot be extended.	Deal slips out of the quarter. Your commission slips with it. If you were counting on this deal for quota attainment, you just made attainment harder.
Customer perception	The prospect experiences a broken or underwhelming product when the real issue was an unverified config. The platform is judged on a state it was never meant to run in.	A skeptical customer is harder to close. Lower close probability is lower expected commission on this deal — and a logo that won't reference you on the next one.
Deal risk	A POC that does not demonstrate true capability stalls or loses outright. We lose on a self-inflicted error, not on product merit.	No closed deal = no commission. Enough of those and you miss quota — which means no accelerators on top of the deals you do close. The math compounds against you fast.

Bottom line

Your variable comp is built on closed deals. Every step in this manual exists to raise the close probability of the POC in front of you. Skipping any of them lowers it — directly, every time — and the cost shows up in your paycheck.

8. Golden State & Best-Practice Configuration

The golden state is the reference configuration that every POC agent is validated against. It is derived from the default Helm chart published in the Komodor repository.

The default Helm values live in:

<https://github.com/komodorio/helm-charts/blob/master/charts/komodor-agent/values.yaml>

9. Known Tooling Gaps & Constraints

This ROE assumes the Solutions team can actually perform the validation and steady-state work it requires. Today there are gaps in tooling and access that limit who on the team can execute it. These are flagged here because closing them requires investment or a pricing decision — they are not something an individual SE can resolve.

9.1 LaunchDarkly (LD) access is limited across the Solutions Team

Only a few members of the Solutions Team currently have LD update access. That concentrates the work this ROE describes on a small number of people and leaves most of the team unable to perform it independently.

How to address:

- A full list of FF's and their designation can be found here:
<https://mockador.io/report/komodor-feature-flag-catalog>
- *Follow the steps below to “impersonate”:*
- *Due to limitation of seats in LaunchDarkly on our current billing plan, and since you all are assigned to READER role there, we’ve changed the way you should access LD:*
- *Here’s the link to 1Password, where there is the google username and password.*
- <https://share.1password.com/s#YbStVJxIYo6JYWQjZT7bJIZMaX0odTF6NvBFOBth4xl>
- *You should use Incognito mode in:*
- *your browser>go to gmail>9 dots at the top right corner>scroll down to the launchdarkly icon*

Increasing LD licenses would force us off our current legacy pricing and roughly 4x our cost. The decision is therefore not simply “buy more seats” — it is how to give the broader Solutions Team the access this ROE assumes without triggering that cost increase.

9.2 No R/O access to reporting database

As of July 8, 2026, reporting database access is not available due to migrating of Postgres to Clickhouse.

How to address:

- A full list of FF's and their designation can be found here:
<https://mockador.io/report/komodor-feature-flag-catalog>
- *Follow the steps below to “impersonate”:*
- *Due to limitation of seats in LaunchDarkly on our current billing plan, and since you all are assigned to READER role there, we’ve changed the way you should access LD:*
- *Here’s the link to 1Password, where there is the google username and password.*
- <https://share.1password.com/s#YbStVJxIYo6JYWQjZT7bJIZMaX0odTF6NvBFOBth4xl>
- *You should use Incognito mode in: your browser*
 - >go to gmail*
 - >9 dots at the top right corner*
 - >scroll down to the launchdarkly icon*

9.3 Additional gaps

10. Open Items to Complete Before Final

The following items are placeholders to be filled in to make this document complete and final. Reference numbers map to the sections above.

Ref	Item to provide	Owner
2.1	Helm Images - https://github.com/nikhilsbhat/helm-images	<i>Phil</i>
8	https://github.com/komodorio/helm-charts/blob/master/charts/komodor-agent/values.yaml	<i>Phil</i>

Ref	Item to provide	Owner
4.2	Confirm which Klaudia inventory / golden-comparison prompts are supported today vs roadmap.	[product team]
6.1	Specific symptoms / error patterns (incl. Klaudia signals) that trigger escalation.	[owner]
6.1	Internal test cluster spec for isolation (match customer K8s version, or representative?).	[owner]
6.2	P0 == ASAP, p1 == next business day	Komodor Technical support
Title	Document owner name & title.	[you]

10. Appendix

Links

All Komodor Helm Chart Images for Komodor Agent as of June 2026

- *alpine:latest*
- *public.ecr.aws/komodor-public/telegraf:v2.0.59-alpine*
- *public.ecr.aws/komodor-public/komodor-agent:0.2.220*
- *public.ecr.aws/komodor-public/komodor-agent:0.2.220*
- *public.ecr.aws/komodor-public/komodor-otel-collector:0.1.10*
- *public.ecr.aws/komodor-public/komodor-agent:0.2.220*
- *public.ecr.aws/komodor-public/komodor-agent:0.2.220*
- *public.ecr.aws/komodor-public/komodor-agent:0.2.220*
- *public.ecr.aws/komodor-public/komodor-agent:0.2.220*
- *public.ecr.aws/komodor-public/telegraf-windows:v2.0.59*
- *public.ecr.aws/komodor-public/komodor-agent-windows:0.2.220*
- *public.ecr.aws/komodor-public/komodor-agent-windows:0.2.220*
- *public.ecr.aws/komodor-public/admission-controller:0.1.64*
- *public.ecr.aws/komodor-public/komodor-agent:0.2.220*
- *public.ecr.aws/komodor-public/komodor-agent:0.2.220*
- *public.ecr.aws/komodor-public/telegraf:v2.0.59-alpine*
- *public.ecr.aws/komodor-public/komodor-agent:0.2.220*
- *public.ecr.aws/komodor-public/komodor-agent:0.2.220*